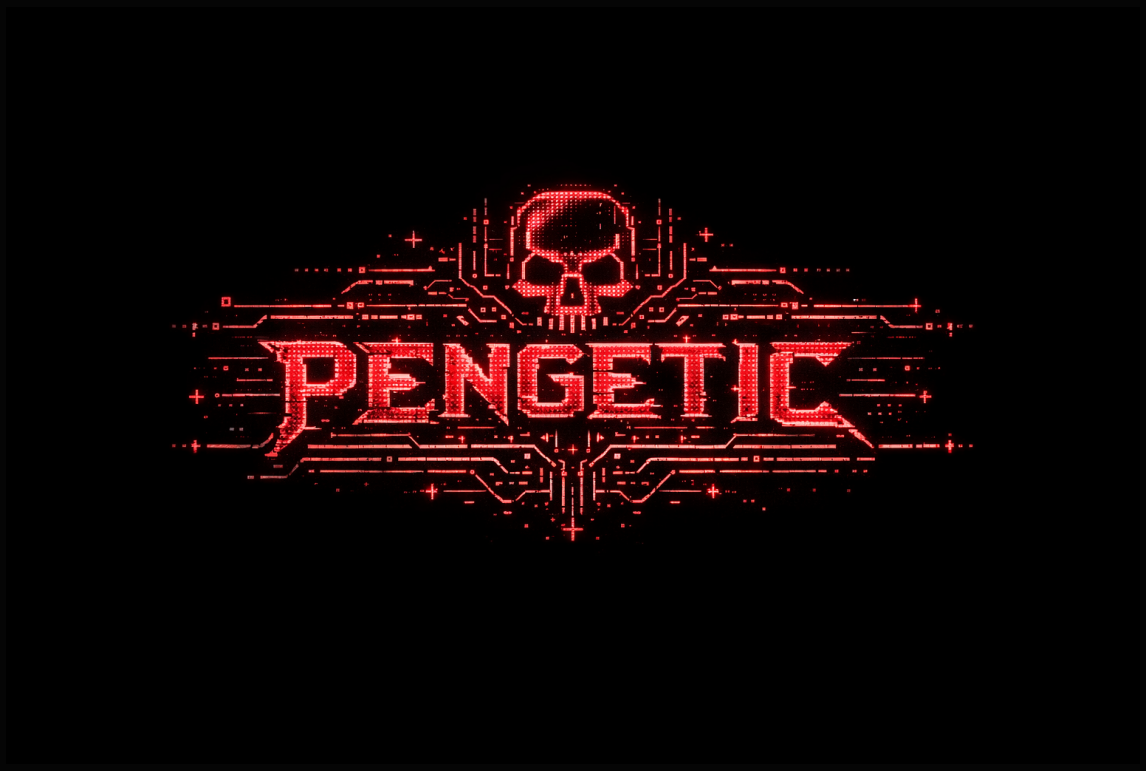

Pengetic User Manual

Complete operating guide for the local-first defensive web assessment platform

Current release: v2.0.2

Prepared: 2026-03-22

Authorized use only. This manual covers installation, the GUI, the CLI, scope packages, approvals, the planner, the orchestrator, reports, persistence, and troubleshooting.

Pengetic is scope-bound by design and does not permit unrestricted shell access or autonomous exploit execution.

Table of Contents

The headings below are collected automatically from the manual content. The section numbers mirror the document chapters.

What This Manual Covers

This manual explains how to install, start, operate, troubleshoot, and extend Pengetic from the point of view of a normal authorized user. It covers:

- Fresh install from a release source zip or from a git checkout
- The backend service and the GUI
- The CLI commands
- Scope package authoring and validation
- Assessment planning, execution, approvals, and reports
- The local Ollama planner and orchestrator
- The SQLite data model and on-disk workspace layout
- Common problems and how to fix them

Pengetic is designed for authorized defensive work only. It does not execute offensive exploit chains, brute force, credential attacks, persistence, or destructive testing.

1. What Pengetic Is

Pengetic is a local-first defensive web assessment platform. It combines:

- A scope-gated assessment engine
- A FastAPI backend
- A React + Tailwind front end
- SQLite persistence for runs, findings, approvals, and artifacts
- A local Ollama-backed planner service

The platform is closed by default:

- No valid scope, no run
- No non-passive action without approval
- No unrestricted shell access for the LLM
- No execution outside the validated scope

If you are new to the tool, the important idea is simple: Pengetic helps you inspect and manage an authorized assessment. It is not a generic attack framework.

2. What Pengetic Is Not

Pengetic is not:

- A brute-force tool
- A payload launcher
- An exploit chain runner
- A persistence framework
- A social engineering toolkit
- A command shell for the LLM

If you need any of those behaviors, Pengetic is intentionally not the right tool.

3. System Requirements

Recommended baseline:

- Python 3.12
- A modern Node.js LTS release
- Git if you are working from a repository clone
- A Chromium-based browser, Firefox, or Safari for the GUI
- Ollama if you want the local planner to return live LLM suggestions
- If you already have Ollama in WSL with `qwen2.5-coder:14b`, Pengetic can use it as long as the OpenAI-compatible endpoint is reachable from the environment running Pengetic

Windows users can run Pengetic from PowerShell. macOS and Linux users can use the same commands with the platform-specific virtual environment activation step.

4. First-Time Setup

The safest first-run flow is:

1. Create a Python virtual environment.
2. Install Pengetic in editable mode.
3. Install and build the front end.
4. Start the backend with `pengetic serve` or `python -m pengetic serve`.
5. Open the UI in the browser.

Fresh install from source

```
python -m venv .venv

# Windows PowerShell
.venv\Scripts\Activate.ps1

# macOS or Linux
source .venv/bin/activate

python -m pip install -e .
cd frontend
npm install
npm run build
cd ..
python -m pengetic serve
```

Optional: Use Ollama in WSL

If your Ollama service is running inside WSL, keep Pengetic on the host and point it at the reachable Ollama endpoint.

1. Start Ollama in WSL.
2. Verify the endpoint answers the OpenAI-compatible API from the same machine where Pengetic runs.
3. Set the planner variables before starting Pengetic.

```
export OLLAMA_BASE_URL=http://127.0.0.1:11434/v1
export OLLAMA_MODEL=qwen2.5-coder:14b
```

If `127.0.0.1` is not the address that reaches your WSL Ollama service, use the reachable WSL address instead. The important requirement is that Pengetic can call `/chat/completions` on the configured base URL.

Fresh install from a GitHub source zip

If you downloaded the release source archive, unzip it, then run the same steps from the extracted folder. The important detail is that the front end must be built once before `pengetic serve` can serve the real UI.

What a successful first launch looks like

- The backend prints that it is serving the built front end from `frontend/dist`
- Opening `http://127.0.0.1:8000/` shows the Pengetic UI
- The dashboard shows zero runs, zero findings, and zero pending approvals on a clean database

- The logo appears in the left side of the UI shell

If `frontend/dist` does not exist, the backend shows a setup hint page instead of a blank page.

5. Repository Layout

Key paths:

```
src/pengetic/          FastAPI backend, CLI, state machine, storage, LLM planner
src/scopeguard/        Assessment engine and policy core reused by Pengetic
frontend/              React + Tailwind GUI
frontend/public/       Static logo and browser icon assets
docs/                  Architecture and policy documentation
examples/              Demo scope packages
tests/                 Regression tests
data/                  SQLite database and app state
artifacts/             Runs, reports, evidence, and audit logs
```

The front-end logo asset lives at:

```
frontend/public/pengetic-logo.png
```

That file is copied into the production build and served directly by the backend.

6. Main User Workflows

There are four common ways to use Pengetic:

1. Use the GUI for an interactive assessment.
2. Use the CLI for scripted or repeatable work.
3. Use the planner and orchestrator to drive a run step by step.
4. Export reports and evidence for review or recordkeeping.

The GUI and CLI both operate on the same underlying data. If you upload a scope in the GUI, the backend stores it for later runs. If you create a run in the CLI, the GUI will show it as soon as the database reflects the change.

7. Security Model and Safety Rules

The safety model is central to Pengetic.

- `PASSIVE_SAFE` actions can run automatically if the scope allows them.
- `LOW_RISK_ACTIVE` actions require explicit approval.
- `HIGH_RISK_ACTIVE` actions require explicit approval and should be used sparingly.
- `FORBIDDEN` actions never execute.

Additional rules:

- Only assets listed in scope are considered authorized.
- Approvals are bound to the exact action id and scope fingerprint.
- The LLM may summarize state and recommend the next allowed step, but it cannot execute arbitrary shell commands.
- The orchestrator stops when no useful actions remain, when approval is required, when rate limits block progress, or when the max step count is reached.

8. The Pengetic State Machine

Pengetic tracks assessment progress with these states:

- `idle`
- `scope_uploaded`
- `scope_validated`
- `plan_ready`
- `running`
- `awaiting_approval`
- `completed`
- `failed`

Practical meaning:

- `idle` means no active assessment context exists.
- `scope_uploaded` means a scope file has been ingested.
- `scope_validated` means the scope passed validation.
- `plan_ready` means a plan exists and can be reviewed.
- `running` means actions are executing.
- `awaiting_approval` means the run is paused for a gated action.
- `completed` means the current run finished without outstanding approvals.
- `failed` means something blocked the assessment, such as validation or runtime error.

The GUI surfaces these states as badges and empty-state messages. The orchestrator and API also use them in responses and stored records.

9. Quick Start From the GUI

Use this path if you want to work visually:

1. Start the backend with `pengetic serve`.
2. Open the browser at the local URL the backend prints.
3. Upload a scope package in the Scope view.
4. Review the generated plan in the Plan view.
5. Start a run from the dashboard or the run panel.
6. Watch the live Run view for events, evidence, and findings.
7. Approve any active action only if your authorization covers it.
8. Review the report and export it when finished.

The left sidebar moves you between:

- Overview
- Scope
- Plan
- Run
- Approvals
- Reports
- Planner

10. Quick Start From the CLI

The CLI is useful when you want repeatable runs or want to script the workflow.

```
pengetic validate-scope examples/scope.demo.yaml
pengetic plan examples/scope.demo.yaml
pengetic approve examples/scope.demo.yaml active-login-surface-probe
pengetic run examples/scope.demo.yaml
pengetic report examples/scope.demo.yaml
pengetic serve
pengetic paths
```

The same commands are available through `python -m pengetic ...` if you prefer module execution.

11. CLI Reference

`pengetic validate-scope <scope.yaml>`

Use this to verify that a YAML scope file is valid before you run anything else.

What it does:

- Loads and validates the YAML
- Checks that the scope has the required fields
- Prints the primary domain, base URL, and authorized hosts
- Exits with a non-zero status if validation fails

Use this first whenever you edit a scope file.

`pengetic plan <scope.yaml>`

Use this to build a risk-labelled plan for a validated scope.

Useful options:

- `--artifacts-root` to change where plan files are written
- `--profile` to choose the runtime profile

What it writes:

- `plan.json`
- `scope.json`

What it prints:

- The plan file location
- One line per action showing the action id, risk class, scope status, and whether approval is required

`pengetic approve <scope.yaml> <action-id>`

Use this to record an explicit approval for a non-passive action.

Useful options:

- `--approved-by` to identify the reviewer
- `--note` to explain why the action was approved
- `--artifacts-root` to point at a different workspace

Important behavior:

- Passive-safe actions do not need approval
- Approval records are tied to the scope fingerprint
- Approving the wrong action id will not unlock a different action

`pengetic run <scope.yaml>`

Use this to execute the assessment engine.

Useful options:

- `--profile` to set the runtime profile
- `--include-approved-active` to allow approved active actions to run
- `--manual-notes` to attach analyst notes to the run
- `--artifacts-root` to store outputs in a different location

What it prints:

- Run id
- Report path
- Finding summary

`pengetic report <scope.yaml>`

Use this to read the stored report for a run.

Useful options:

- `--run-id` to choose a specific run
- `--output` to write the report to a file
- `--artifacts-root` to read from a custom workspace

If you do not supply `--output`, the report is printed to the console.

`pengetic serve`

Use this to start the FastAPI backend and serve the production front end.

Useful options:

- `--host`
- `--port`
- `--reload/--no-reload`

Important behavior:

- If `frontend/dist` exists, Pengetic serves the real UI.
- If the build is missing, Pengetic shows a clear setup page instead of a blank screen.
- Real static files such as JavaScript modules and the logo are served as actual files, not as HTML fallback.

`pengetic paths`

Use this to print the current runtime paths for:

- Repository root
- Data directory
- SQLite database path
- Artifact root
- Front-end build output

12. Runtime Profiles

Pengetic uses three runtime profiles:

- `passive-only`
- `report-only`
- `lab-safe`

Behavior by profile:

- `passive-only` runs passive checks and queues active actions for approval.
- `report-only` suppresses passive execution and focuses on planning and reporting.
- `lab-safe` allows passive checks and approved active checks.

The profile is part of the assessment context. Choose the smallest profile that matches your authorization.

13. Scope Package Authoring

The scope package is a versioned YAML file. It defines what you are allowed to assess.

Required top-level fields:

- `version`
- `name`
- `primary_domain`
- `base_url`
- `allowed_subdomains`
- `allowed_urls`
- `out_of_scope_assets`
- `login_areas_allowed`
- `apis_allowed`
- `tool_allowlist`
- `rate_limits`
- `testing_window`
- `authorization_note`
- `contacts`

Optional field:

- `notes`

Example scope

```
version: 1
name: Demo Assessment
primary_domain: demo.example
base_url: https://demo.example
allowed_subdomains:
  - app.demo.example
allowed_urls:
  - https://demo.example/
  - https://demo.example/login
out_of_scope_assets:
  - admin.demo.example
login_areas_allowed:
  - /login
apis_allowed:
  - /api
tool_allowlist:
  - header-review
  - tls-review
  - robots-fetch
  - sitemap-fetch
  - route-inventory
  - tech-fingerprint
  - manual-review
rate_limits:
  max_requests_per_minute: 60
  max_concurrent_requests: 2
  delay_seconds_between_requests: 0.5
testing_window:
  start: 2026-03-22T00:00:00Z
  end: 2026-03-29T23:59:59Z
authorization_note: Authorized by the site owner for defensive assessment only.
contacts:
  - security@example.com
notes: Demo scope for local testing.
```

Validation rules

Pengetic rejects scope files that violate any of the following:

- Wildcard hosts or routes
- `base_url` hosts outside the primary domain or allowed subdomains
- `allowed_urls` that point at hosts outside the authorized set
- `login_areas_allowed` or `apis_allowed` entries whose hosts are not in scope
- Empty `tool_allowlist`
- Missing `authorization_note`
- `testing_window` where the start is after the end

The scope loader normalizes lists, routes, and hosts so the internal representation stays consistent.

Practical advice

- Keep the scope explicit.
- List only assets you are actually allowed to test.
- Include a short authorization note.
- Add contacts so the assessment can be traced back to the approved owner.

- Use the smallest possible allowlist.

14. Tool Catalog

Passive-safe tools

- `header-review` - reviews HTTP headers and cookies for common security posture gaps
- `tls-review` - inspects HTTPS transport and certificate posture
- `robots-fetch` - fetches `robots.txt`
- `sitemap-fetch` - fetches `sitemap.xml`
- `route-inventory` - builds a public route inventory from visible links and discovery files
- `tech-fingerprint` - performs passive technology fingerprinting from headers and HTML markers
- `manual-review` - records analyst notes as evidence

Gated active tools

- `approved-login-surface-probe` - approved login surface check
- `approved-api-surface-probe` - approved API surface check
- `approved-rate-limit-probe` - approved rate limit check

The active tools are not meant for arbitrary exploitation. They remain scope-bound and approval-gated.

15. Approvals

Approvals exist so that non-passive actions never run automatically.

How approvals work:

1. The plan marks an action as approval-required.
2. The GUI shows that action in the approval queue.
3. You record approval with a reviewer name and note.
4. The stored approval is matched against the exact action id and scope fingerprint.
5. The run can then resume and include approved active actions if the profile allows it.

Important guidance:

- Do not approve actions outside your authorization
- Do not reuse approvals across unrelated scopes
- Do not approve a step just because it is available
- Always read the action objective and expected evidence first

The approval queue only exposes non-passive actions.

16. The Plan Viewer

The plan is a structured list of actions generated from the validated scope.

Each action includes:

- `action_id`
- title
- objective
- target
- tool id
- risk classification
- expected evidence
- whether approval is required
- whether the tool is allowed by scope
- optional notes

In the GUI, the plan viewer makes it easy to see:

- Which actions are passive-safe
- Which actions are gated
- Which actions are blocked by the scope allowlist
- What evidence each action is expected to produce

Read the plan before starting a run. It tells you what Pengetic will do and what it will wait for.

17. The Run View

The live Run view is where assessment execution becomes visible.

It shows:

- The current run state
- The action list
- Live events and status messages
- Evidence paths
- Findings
- Stored artifacts
- Approval status

Typical run statuses:

- `queued`
- `pending-approval`
- `approved`
- `executed`
- `skipped`
- `blocked`

What to watch for:

- A passive action should move to `executed`
- A gated active action should stop at `pending-approval`
- A blocked action should remain outside execution
- Findings should appear after the relevant passive or approved action completes

The run view updates live using the server event stream when the assessment is running.

18. Reports and Export

Every run can produce a Markdown report.

The report includes:

- Executive summary
- Scope confirmation
- Methodology
- Findings
- Execution summary
- Evidence
- Limitations
- Run metadata

How to use reports:

- View them in the GUI
- Copy the markdown from the report panel
- Export them through the GUI
- Save them with `pengetic report --output <file>`

The report is stored with the run and can also be exported from the API.

19. The Planner and Orchestrator

Pengetic includes a local planner service backed by Ollama.

Common local setup:

- Ollama running in WSL
- Model selected as `qwen2.5-coder:14b`
- `OLLAMA_BASE_URL` set to the reachable OpenAI-compatible endpoint
- `OLLAMA_MODEL` set to the installed model name

Planner purpose:

- Summarize the current assessment state
- Explain the current findings and progress
- Recommend the next allowed step from the approved plan

Planner limitations:

- It cannot execute arbitrary shell commands
- It cannot bypass approvals
- It cannot leave scope
- It cannot invent new tools beyond the registry

Environment variables:

- `OLLAMA_BASE_URL` - OpenAI-compatible base URL, default `http://127.0.0.1:11434/v1`
- `OLLAMA_MODEL` - model name, default `llama3.1`

Example for a WSL-hosted Ollama service with the Qwen 14B coding model:

```
OLLAMA_BASE_URL=http://127.0.0.1:11434/v1
OLLAMA_MODEL=qwen2.5-coder:14b
```

The orchestrator loop:

- Reads the current run state
- Reads findings, completed steps, evidence, remaining actions, and rate-limit state
- Uses the planner to choose the next allowed step
- Executes only from the existing tool registry
- Stores each decision and result in SQLite
- Stops when no further useful actions remain

You can call the orchestrator from the API or integrate it into a GUI workflow.

20. Backend API Overview

The GUI uses the same backend API you can call directly.

Health and summary

- `GET /api/health` - service health
- `GET /api/dashboard` - dashboard snapshot

Scope and plan

- `GET /api/scopes/current` - current active scope
- `GET /api/scopes` - all stored scopes
- `POST /api/scopes/upload` - upload and validate a scope
- `GET /api/plans/current` - current active plan

Runs

- `GET /api/runs` - list runs
- `POST /api/runs` - start a run
- `GET /api/runs/{run_id}` - run detail
- `GET /api/runs/{run_id}/actions` - run actions
- `GET /api/runs/{run_id}/events` - run event log
- `GET /api/runs/{run_id}/stream` - live event stream
- `POST /api/runs/{run_id}/resume` - resume a paused run

Approvals

- `GET /api/runs/{run_id}/approvals` - approvals for a run
- `POST /api/approvals` - record a new approval

Findings, artifacts, and reports

- `GET /api/runs/{run_id}/findings` - findings for a run
- `GET /api/runs/{run_id}/artifacts` - artifacts for a run
- `GET /api/reports/{run_id}` - report metadata and text
- `GET /api/reports/{run_id}/export` - download the report markdown

Planner and orchestrator

- `POST /api/llm/planner` - ask the local planner for a summary and suggestion
- `POST /api/runs/{run_id}/orchestrate` - run the orchestrator loop
- `GET /api/runs/{run_id}/orchestrator/steps` - review stored orchestrator decisions

Front-end serving

- `GET /` - serves the built UI or the setup hint page

- `GET /assets/{asset_path}` - serves built static assets
- `GET /{path:path}` - serves app routes and static public files safely

21. Environment Variables

Pengetic recognizes these environment variables:

- `PENGETIC_ROOT` - override the repository root used for runtime paths
- `PENGETIC_DATA_DIR` - override the data directory
- `PENGETIC_ARTIFACTS_DIR` - override the artifact root
- `PENGETIC_FRONTEND_DIST` - override the front-end build output path
- `PENGETIC_API_HOST` - override the backend bind host
- `PENGETIC_API_PORT` - override the backend port
- `OLLAMA_BASE_URL` - override the planner service URL
- `OLLAMA_MODEL` - override the planner model
- `VITE_PENGETIC_API_BASE` - override the API base URL in the front-end dev build

If you do not set them, Pengetic uses sensible local defaults.

22. SQLite Database and Files

Default data path:

```
data/pengetic.sqlite3
```

Database tables:

- `settings`
- `scopes`
- `plans`
- `runs`
- `run_actions`
- `approvals`
- `findings`
- `artifacts`
- `events`
- `llm_recommendations`
- `orchestrator_steps`

Typical workspace layout for one assessment:

```
artifacts/&lt;assessment-id&gt;/
  plan.json
  approvals.jsonl
  scope.json
  runs/
    &lt;run-id&gt;/
      audit.jsonl
      evidence/
      report.md
```

The assessment id is derived from the scope name and scope fingerprint. The run directory is derived from the run id or a timestamp slug.

23. Practical End-to-End Workflows

Workflow A: Start from a scope file and run a passive assessment

1. Validate the scope.
2. Build the plan.
3. Review the plan in the GUI.
4. Start the run with the `passive-only` profile.
5. Wait for passive-safe actions to execute.
6. Inspect findings and evidence.
7. Export the report.

Workflow B: Handle a gated active action

1. Start a run.
2. Stop when the queue reaches `awaiting_approval`.
3. Read the action objective and expected evidence.
4. Confirm the action is explicitly authorized.
5. Approve the action.
6. Resume the run.
7. Review the completed evidence and findings.

Workflow C: Use the local planner

1. Start Ollama locally or inside WSL.
2. Configure `OLLAMA_BASE_URL` and `OLLAMA_MODEL` if needed.
3. For your ready-to-use model, set `OLLAMA_MODEL=qwen2.5-coder:14b`.
4. Open the Planner view in the GUI or call the planner API.
5. Read the suggested next allowed step.
6. If the suggestion points at an approved action, resume or orchestrate the run.

Workflow D: Script the CLI

1. Validate the scope.
2. Generate the plan.
3. Approve exact active steps if needed.
4. Run the assessment.
5. Export the report.

24. Troubleshooting

The UI is blank

Most likely cause:

- The front end was not built before starting the backend

Fix:

```
cd frontend
npm install
npm run build
cd ..
python -m pengetic serve
```

JavaScript module MIME-type errors appear in the browser

Most likely cause:

- The backend is falling back to HTML for a missing asset path
- The browser is holding an older cached build

Fix:

- Rebuild the front end
- Restart `pengetic serve`
- Hard refresh the browser
- Confirm `/assets/*.js` returns `text/javascript`

The logo does not appear

Most likely cause:

- `frontend/public/pengetic-logo.png` is missing
- The build is stale

Fix:

- Confirm the file exists
- Rebuild the front end
- Restart the backend

Scope upload fails

Most likely cause:

- A field is missing
- A wildcard was used in a host or route
- The base URL host is not covered by the scope

Fix:

- Run `pengetic validate-scope <file>`
- Read the validation error

- Correct the YAML and upload it again

The planner has no response

Most likely cause:

- Ollama is not running or the WSL endpoint is not reachable
- The base URL is wrong
- The model is unavailable

Fix:

- Start Ollama
- If it runs in WSL, confirm the endpoint is reachable from the Pengetic process
- Check `OLLAMA_BASE_URL`
- Check `OLLAMA_MODEL`
- Try `OLLAMA_MODEL=qwen2.5-coder:14b` if that is the installed model
- Try the planner again

A run is stuck at awaiting approval

Most likely cause:

- The next action requires approval and none has been recorded yet

Fix:

- Read the action objective
- Confirm the approval is in scope
- Use the GUI approval queue or `pengetic approve`

The backend port is already in use

Most likely cause:

- Another Pengetic process is already running
- A different service is bound to the same port

Fix:

- Stop the old process
- Or start Pengetic with `--port` set to a different value

25. Operator Checklist

Before you start an assessment, confirm:

- You have explicit authorization
- The scope file is valid
- The base URL and subdomains are correct
- The tool allowlist is minimal and accurate
- The rate limits are acceptable
- The testing window is active
- The front end has been built
- Ollama is available if you want local planning
- You know which actions will require approval

Before you approve a non-passive action, confirm:

- The action id matches the plan
- The action is explicitly authorized
- The scope fingerprint matches the current scope
- You understand the expected evidence

Before you export a report, confirm:

- The run completed or paused for a reason you understand
- The findings look correct
- No secrets or personal data were exposed in evidence
- The report reflects the actual assessment

26. File and Artifact Reference

Useful runtime files:

- `data/pengetic.sqlite3`
- `artifacts/<assessment-id>/plan.json`
- `artifacts/<assessment-id>/approvals.jsonl`
- `artifacts/<assessment-id>/scope.json`
- `artifacts/<assessment-id>/runs/<run-id>/audit.jsonl`
- `artifacts/<assessment-id>/runs/<run-id>/evidence/`
- `artifacts/<assessment-id>/runs/<run-id>/report.md`

Useful source files:

- `src/pengetic/api.py`
- `src/pengetic/cli.py`
- `src/pengetic/llm.py`
- `src/pengetic/agent.py`
- `src/pengetic/storage.py`
- `src/pengetic/state.py`
- `frontend/src/App.tsx`
- `frontend/index.html`
- `frontend/public/pengetic-logo.png`

27. Final Notes

Pengetic is meant to be a professional defensive assessment platform. Use it to:

- keep work inside explicit scope
- review risk before execution
- require approval for active actions
- record evidence and findings cleanly
- export reproducible reports

If you keep those rules in mind, the rest of the interface is straightforward:

1. Load scope.
2. Review the plan.
3. Run passive checks.
4. Approve gated actions when authorized.
5. Export the report.